파이썬 기본 데이터 타입: 숫자와 문자열

이청용 교수
leecy@hansung.ac.kr

강의개요

- **데이터 입력하기**
  - 객체와 변수
  - 변수 이름 규칙
  - 변수 이름 예시
- **숫자형**
  - 숫자형 개요
  - 수학 연산자
  - 숫자형 사칙연산
  - 숫자형 변환
- **문자열**
  - 문자열 개요
  - 문자열 변환
  - 이스케이프 문자(Escape Character)
  - 문자열 결합
  - 문자열 인덱싱
  - 문자열 슬라이싱
  - 문자열 분할
  - 문자열 수정
  - 문자열 확인
  - 문자열 대소문자 변환
  - 문자열 포맷팅

데이터 입력하기

객체와 변수

- Python에서 데이터 저장 방식
  - 모든 숫자, 문자열, 자료구조, 함수, 클래스 등은 객체(object) 형태로 존재
  - 변수는 객체를 참조하는 이름(name) 역할을 수행
- 변수 사용 방법
  - 연산자를 사용하여 변수에 객체를 할당
  - 동일한 변수 이름에 새 값을 할당하면 기존 객체 대신 새로운 객체를 참조

```python
a = 5         # 정수
b = 3.14      # 실수
c = "Hello"   # 문자열
print(a, b, c)
```

```
5 3.14 Hello
```

변수 이름 규칙

- 사용 가능한 문자
  - 소문자 (a ~ z)
  - 대문자 (A ~ Z)
  - 숫자 (0 ~ 9) → 단, 첫 글자로는 사용 불가
  - 언더스코어(_)
- 사용 시 주의사항
  - 숫자로 시작 불가(예: 1a, 95score)
  - 대소문자 구분(예: Data ≠ data)
  - 의미를 쉽게 파악할 수 있는 이름 사용 권장(예: score, student_name)
- 예약어(Reserved Word)는 변수명으로 사용 불가
  - 예: if, for, class, def, return, False, True 등

변수 이름 예시

- 올바른 변수 이름
  - 예: name, score_95, _abc
- 잘못된 변수 이름
  - 1name(숫자로 시작)
  - class(예약어 사용)
  - 1_(숫자로 시작)

```python
student_name = "Tom"    # O
score95 = 95            # O
1st_student = "Kim"     # X (숫자로 시작)
class = "Python"        # X (예약어)
```

Python

```
  Cell In[4], line 3
    1st_student = "Kim"     # X (숫자로 시작)
    ^
SyntaxError: invalid decimal literal
```

숫자형

숫자형 개요

- Python 숫자 자료형
  - 정수형(int): 소수점이 없는 정수 값(예: $123, -10, 0$)
  - 실수형(float): 소수점이 포함된 값(예: $1.23, -3.5$)
  - 지수 표현: 과학적 표기법 사용 가능(예: $4.24e10 \rightarrow 4.24 \times 10^{10}$, $4.24e\text{-}10 \rightarrow 4.24 \times 10^{-10}$)

```python
a = 123
b = -1.23
c = 4.24E10
d = 4.24e-10
```

Python

수학 연산자

- 기본 연산자
  - + (더하기), – (빼기), * (곱하기), / (나누기)
- 특수 연산자
  - // (몫), % (나머지), ** (거듭제곱)

```python
print(5 + 8)
print(90 - 10)
print(4 * 7)
print(7 / 2)
print(7 // 2)
print(7 % 3)
print(3 ** 4)
```

Python

```
13
80
28
3.5
3
1
81
```

사칙연산

- Python은 계산기처럼 다양한 사칙연산 수행 가능
  - 곱셈(*)은 덧셈(+)보다 우선순위가 높음

```python
a = 5
b = 10

print(a + b)
print(a - b)
print(a * b)
print(a + b - a * b)
```

Python

```
15
-5
50
-35
```

사칙연산

- 나눗셈은 부동소수점을 포함하는 경우와 제외하는 두 가지 방식으로 수행
  - / → 부동소수점 결과 포함
  - // → 소수점 제외, 정수 몫만 반환
  - % → 나머지 반환

```python
a = 5
b = 9

print(b / a)
print(b // a)
print(b % a)
```

```
1.8
1
4
```

숫자형 변환

- 정수형 변환: int( )
  - 실수 → 소수점 버리고 정수 반환
  - 불리언 → True=1, False=0
  - 문자열(숫자 형태) → 정수 변환 가능

```python
print(int(True))
print(int(False))
print(int(98.7))
print(int("89"))
```

Python

```
1
0
98
89
```

숫자형 변환

- 실수형 변환: float( )
  - 정수 → 소수점이 있는 실수로 변환
  - 불리언 → True=1.0, False=0.0
  - 문자열(숫자 형태) → 실수 변환 가능

```python
print(float(10))
print(float(True))
print(float(False))
print(float("98.7"))
```

Python

```
10.0
1.0
0.0
98.7
```

문자열

문자열 개요

- Python이 널리 사용되는 이유 중 하나는 강력하고 유연한 문자열 처리 기능
- 문자열 표현 방법
  - 작은따옴표(') 또는 큰따옴표(") 사용 가능
  - 여러 줄 문자열은 작은따옴표(''') 또는 큰따옴표(""") 세 개를 사용

```python
name = 'Hansung'
city = "Seoul"

c = '''Python
Machine Learning
Pandas
Data Analysis'''

d = """Python
Machine Learning
Pandas
Data Analysis"""
```

문자열 변환

- str( ) 함수를 사용하여 다른 데이터 타입을 문자열로 변환 가능
  - 숫자 → 문자열
  - 불리언 → 문자열

```python
print(str(98.6))
print(str(1.0e4))
print(str(True))
```

```
98.6
10000.0
True
```

이스케이프 문자(Escape Character)

- 문자열 안에서 특수한 의미를 가지는 문자
- 자주 사용되는 이스케이프 코드
  - \n : 줄바꿈 (new line)
  - \t : 탭(tab), 공백 추가

```python
print("Hello\nWorld")
print("Name:\tAlice")
```

```
Hello
World
Name:	Alice
```

문자열 결합

- 문자열 결합 방법
  - + 연산자 사용 → 문자열을 이어 붙임
  - 문자열 사이에 공백은 자동으로 추가되지 않으므로 직접 넣어야 함
  - 리터럴 문자열은 차례로 나열해도 자동 결합

```python
greeting = "Hello" + " " + "Python"
print(greeting)

full_name = "Gildong" " " "Hong"
print(full_name)
```

Python

```
Hello Python
Gildong Hong
```

문자열 결합

- print( ) 함수: 여러 변수를 동시에 출력할 경우, 각 값 사이에 공백이 자동으로 삽입
- * 연산자를 사용하여 문자열을 반복 출력 가능

```python
print("Name:", "Alice", "Age:", 25)
print("Python" * 3)
print("-" * 20)
```

```
Name: Alice Age: 25
PythonPythonPython
--------------------
```

문자열 인덱싱

- 특정 위치의 문자 접근 방법
  - 가장 왼쪽 문자 위치는 0, 오른쪽으로 1, 2, … 증가
  - 가장 오른쪽 문자는 −1, 왼쪽으로 −2, −3, … 감소

```python
string = "Python"

print(string[0])
print(string[5])
print(string[-1])
print(string[-6])
```

```
P
n
n
P
```

문자열 인덱싱

- 유효 범위를 벗어나면 오류 발생

```python
print(string[10])
```

Python

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[5], line 1
----> 1 print(string[10])

IndexError: string index out of range
```

문자열 인덱싱

- Python 문자열은 불변(immutable) 자료형 → 특정 위치의 문자를 직접 변경할 수 없음

```python
name = "Henny"
name[0] = "P"
```

Python

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[11], line 2
      1 name = "Henny"
----> 2 name[0] = "P"

TypeError: 'str' object does not support item assignment
```

문자열 인덱싱

- 문자열 변경 방법
  - replace( ) 함수 사용
  - 슬라이싱과 결합

```python
name = "Henny"
print(name.replace("H", "P"))
print("P" + name[1:])
```

Python

```
Penny
Penny
```

문자열 슬라이싱

- 문자열의 일부(substring) 추출 방법
- 구문: [start:end:step]
  - [:] → 전체 문자열
  - [start:] → start 위치부터 끝까지
  - [:end] → 처음부터 end-1 위치까지
  - [start:end] → start부터 end-1 위치까지
  - [start:end:step] → step 간격으로 추출

```python
string = "Python"
print(string[0:2])
print(string[2:])
```

Python

```
Py
thon
```

문자열 슬라이싱

- step 값을 지정하여 일정 간격으로 문자 추출 가능
- step이 음수이면 역순(reverse) 으로 추출

```python
string = "abcdefg"
print(string[::2])

string = "Python"
print(string[::-1])
```

Python

```
aceg
nohtyP
```

문자열 분할

- split( ) 함수: 문자열을 특정 구분자 기준으로 나누어 리스트로 반환
- 구분자를 지정하지 않으면 기본적으로 공백(스페이스, 탭, 줄바꿈) 기준으로 나눔

```python
sentence = "Hello Python World"
print(sentence.split())

data = "apple,banana,orange"
print(data.split(','))
```

Python

```
['Hello', 'Python', 'World']
['apple', 'banana', 'orange']
```

문자열 분할

- join( ) 함수: 문자열 리스트를 하나의 문자열로 결합
- 구분자를 지정하여 결합 가능

```python
words = ['Hello', 'Python', 'World']

result = ' '.join(words)
print(result)

result = ', '.join(words)
print(result)
```

```
Hello Python World
Hello, Python, World
```

문자열 수정

- replace(old, new) : 특정 문자열을 찾아 새 문자열로 대체
- 세 번째 인자에 숫자를 주면 변경 횟수 지정 가능

```python
text = "Hello World"
new_text = text.replace("World", "Python")
print(new_text)

data = "apple, apple, banana"
new_data = data.replace("apple", "orange", 1)
print(new_data)
```

Python

```
Hello Python
orange, apple, banana
```

문자열 확인

- len( ) 함수: 문자열에 포함된 문자의 개수를 반환 → 공백(space)도 문자로 계산
- find(sub) : 지정한 문자열이 처음 등장하는 위치 반환 (없으면 –1)
- rfind(sub) : 지정한 문자열이 마지막으로 등장하는 위치 반환 (없으면 –1)
- count(sub) : 지정한 문자열이 포함된 개수 반환

```python
text = "Hello World"
print(len(text))

text = "apple banana apple"
print(text.find("apple"))
print(text.rfind("apple"))
print(text.count("apple"))
```

Python

```
11
0
13
2
```

문자열 대소문자 변환

- upper( ) : 문자열을 모두 대문자로 변환
- lower( ) : 문자열을 모두 소문자로 변환
- capitalize( ) : 첫 글자만 대문자로 변환
- title( ) : 각 단어의 첫 글자를 대문자로 변환

```python
text = "hello python"

print(text.upper())
print(text.lower())
print(text.capitalize())
print(text.title())
```

Python

```
HELLO PYTHON
hello python
Hello python
Hello Python
```

문자열 포맷팅

- 서식 문자열의 중괄호 { } 자리에 값을 삽입하여 문장을 구성
  - 위치 인덱스 사용: {0}, {1} …
  - 숫자, 문자열, 변수 등 다양한 타입을 전달 가능

```python
name = "Alice"
age = 20

print("안녕하세요, {0}님! 나이는 {1}살입니다.".format(name, age))
```

```
안녕하세요, Alice님! 나이는 20살입니다.
```

수고하셨습니다.